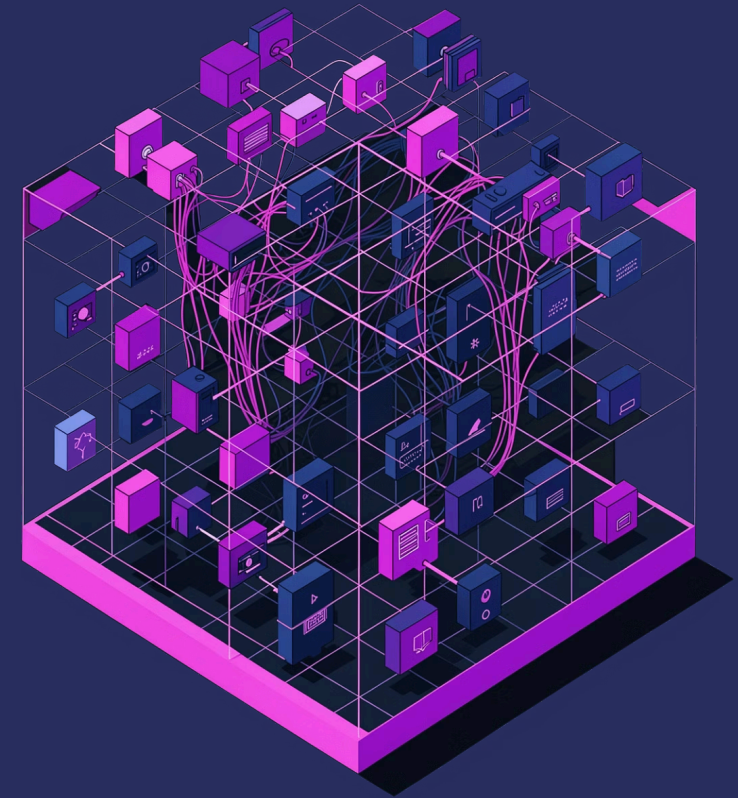
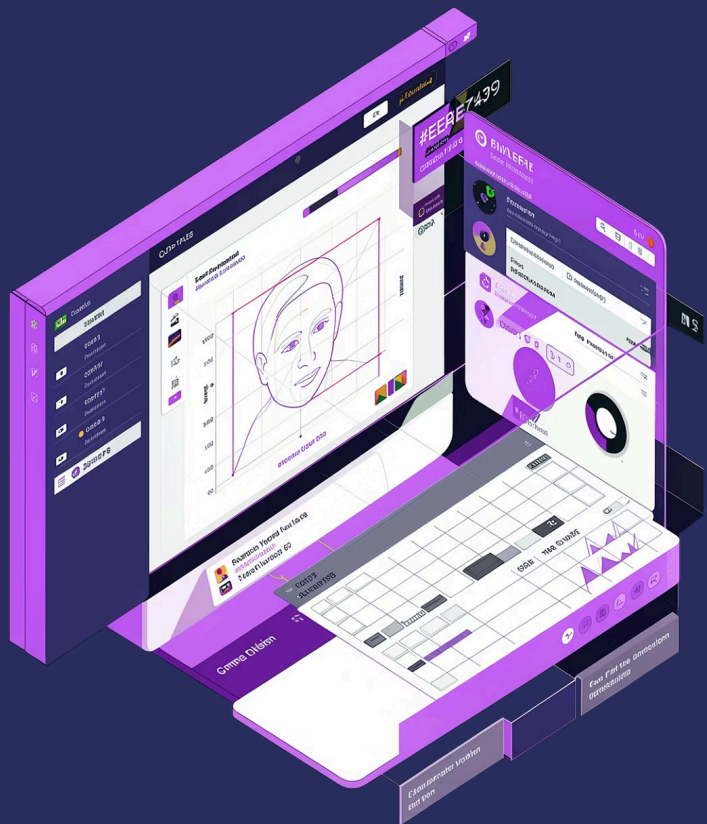


Face Expression Recognition Using CNN

A deep learning computer vision pipeline for classifying facial expressions — built with TensorFlow, Keras, and Google Colab.





Project Overview

Framework

TensorFlow / Keras running on Google Colab

Dataset

jonathanoheix / face-expression-recognition (Kaggle)

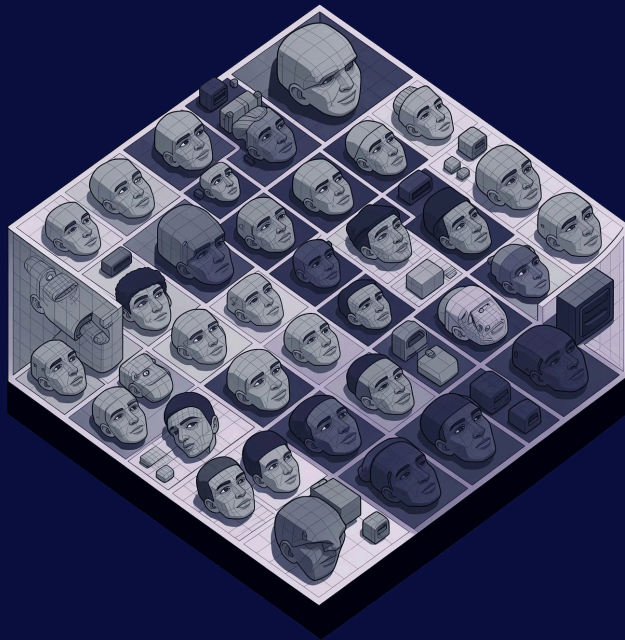
Input

48×48 grayscale face images

Output

7 facial expression classes

Dataset & Preprocessing

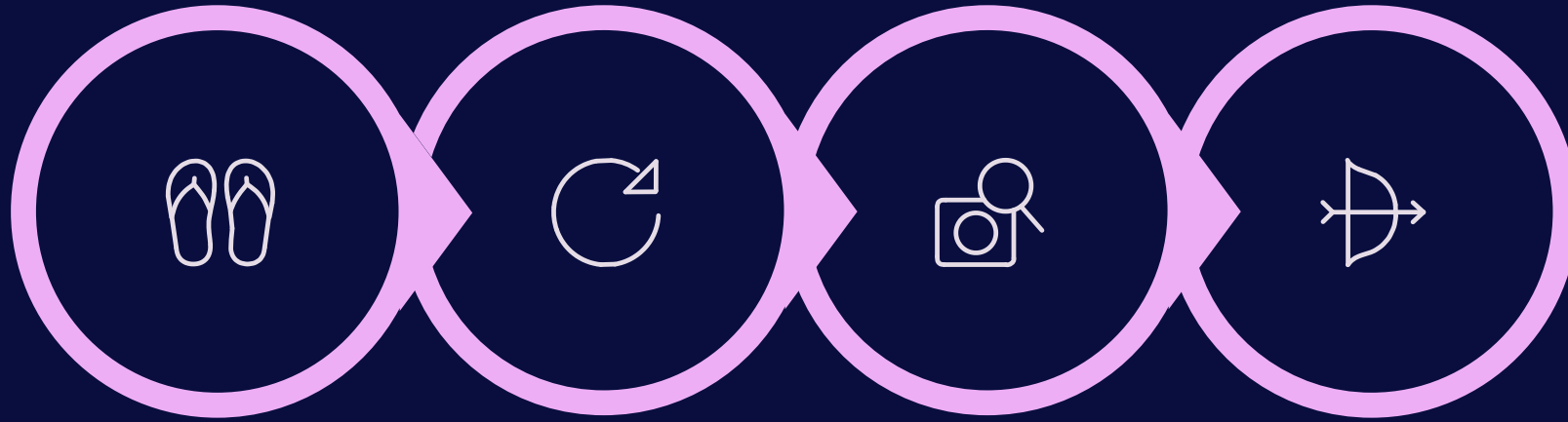


7 Expression Classes

Angry · Disgust · Fear · Happy · Neutral · Sad · Surprise

- **Image Format:** 48×48 grayscale, batch size 64
- **Train / Validation Split:** 80% / 20% with seed 42
- **Normalization:** Rescaling layer (pixel ÷ 255)
- **Data Loading:** `image_dataset_from_directory`

Data Augmentation Pipeline



Flip

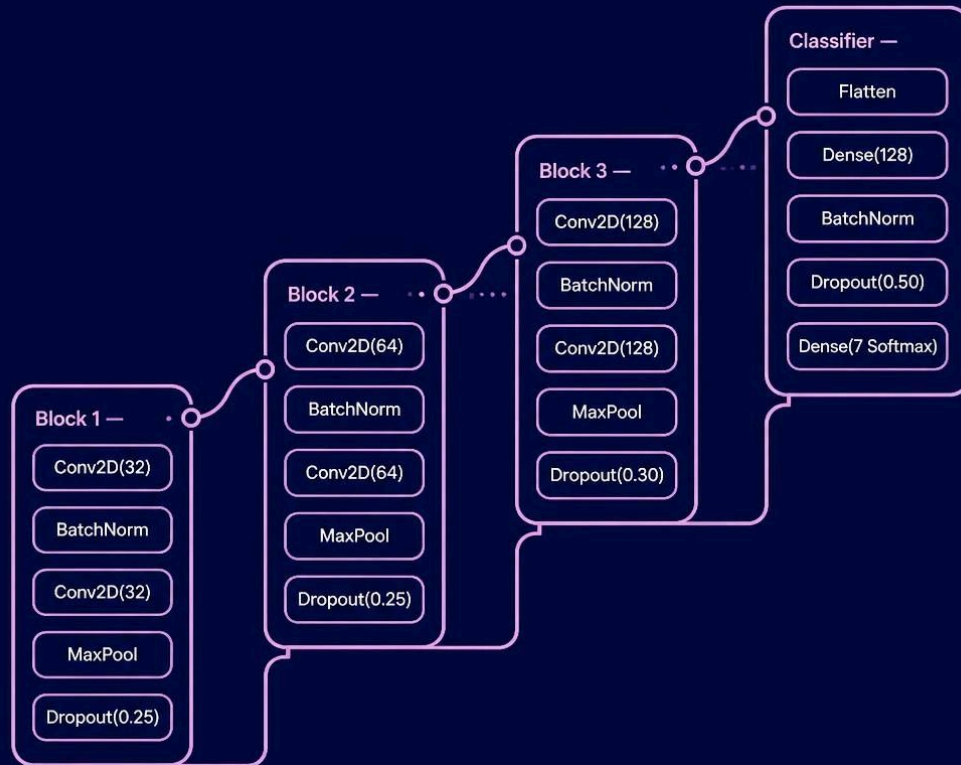
Rotate

Zoom

Translate

Augmentation is applied on-the-fly during training to increase variance and reduce overfitting on the 48×48 grayscale dataset.

CNN Architecture



Design Principles

- Three convolutional blocks with **doubling filter depth** (32 → 64 → 128)
- **Batch normalization** after every Conv2D layer
- **MaxPooling** halves spatial dimensions at each block
- Dropout escalates from **0.25 → 0.30 → 0.50** toward the classifier
- Final Dense(7) with **Softmax** activation for 7-class output

Model Compilation

Optimizer

Adam — learning rate **0.001**

Loss Function

Sparse Categorical Cross-Entropy

Metric

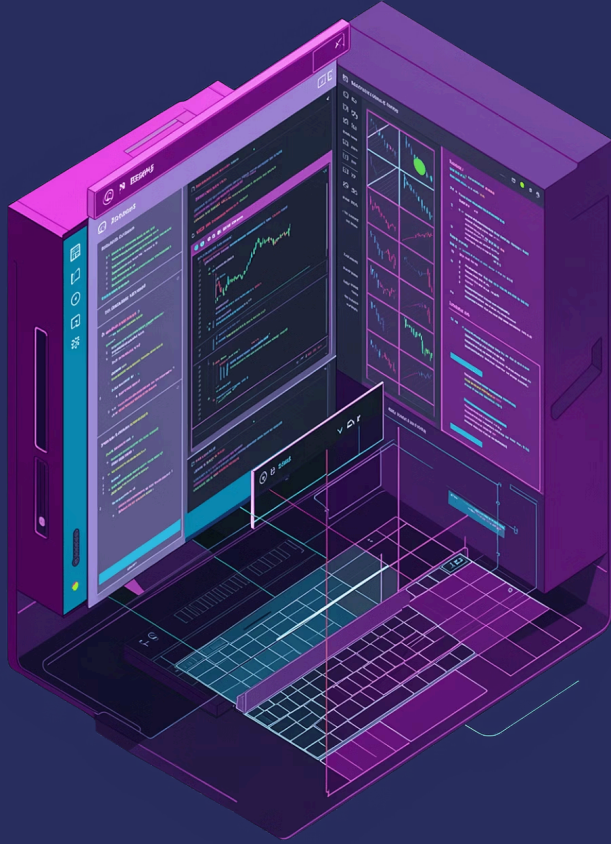
Accuracy tracked on validation set each epoch

LR Scheduler

ReduceLROnPlateau — factor **0.5**, patience **3 epochs**

- The learning rate scheduler halves **lr** whenever validation loss stalls for 3 consecutive epochs, preventing premature convergence.

Training Configuration



40

Epochs

Total training cycles

64

Batch Size

Images per gradient update

1

**Checkpoint
Saved Model**

`best_face_expression_model.keras`

Model Evaluation Metrics



Four-Layer Evaluation Strategy

- **Accuracy & Loss Curves:** Plotted across all 40 epochs to inspect convergence and overfitting
- **Confusion Matrix:** 7×7 grid revealing per-class misclassification patterns
- **Classification Report:** Precision, recall, and F1-score per expression class
- **Visual Validation:** Side-by-side actual vs. predicted expression labels on sample images

Workflow Pipeline



A diagram showing a four-step workflow pipeline. Each step is represented by a horizontal arrow pointing to the right, with the step name written inside. The arrows are stacked vertically and increase in length from top to bottom. The colors of the arrows are a gradient of purple and blue.

Download Data

Preprocess Images

Augment & Train

Evaluate & Deploy

Key Implementation Details



Efficient Data Loading

Uses `image_dataset_from_directory` for streamlined pipeline construction directly from folder structure.



Batch Normalization

Applied after every Conv2D to reduce internal covariate shift and stabilize training dynamics.



Dropout Regularization

Rates of 0.25–0.50 progressively increase toward the classifier to prevent overfitting.



Checkpoint + Inference

Best weights auto-saved during training. Model supports inference on newly uploaded images post-training.